Course: CS213 Object Oriented Programming (2026/2027 First Semester)

Faculty: Faculty of Computers and Artificial Intelligence, Cairo University

Instructor: Dr. Mohammad El-Ramly

Assignment 2 part 1

Team Member Name	Student ID	Section
Andrew Fawzy Mina	20250092	S19,S20
Samy Atef Masoud	20250277	S29,S30
Aly Sameh Mohamed	20250398	S19,S20
Omar Ashraf Mostafa Helal	20250422	S29,S30

Work breakdown

Andrew Fawzy Mina

Video , PDF , 3rd Design

Samy Atef Masoud

Program 2, 1st AI Design

Ali Sameh Mohamed

Final and Initial Designs

Omar Ashraf Mostafa Helal

Program 1 , 2nd design

introduction

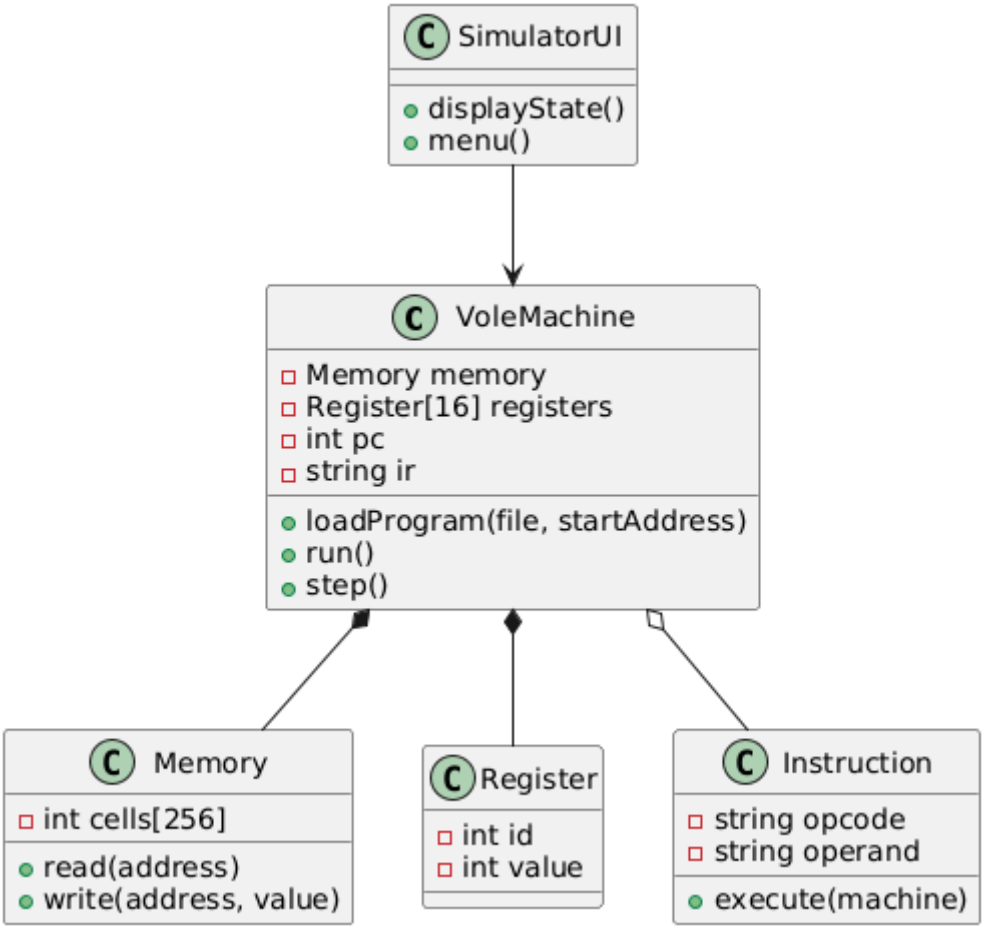
The **Vole Machine** is an educational, simplified virtual computer designed to help students understand foundational computer architecture and low-level execution mechanics. Rather than utilizing complex modern hardware architectures, the Vole machine abstracts computer components into a streamlined model consisting of:

- **16 General-Purpose Registers** for fast data manipulation.
- **256 Bytes of Main Memory** for storing instructions and data.
- **A Central Processing Unit (CPU)** responsible for the fetch-decode-execute cycle, handling arithmetic operations, and managing program flow.

By sequentially processing short instruction sets—loading values, performing arithmetic, and committing results—the simulator provides a transparent environment for learning object-oriented software design principles alongside computer organization fundamentals.

Initial Design

This PlantUML diagrams show how the code is organized into different parts and how the system design improves step by step. The design connects a user menu (SimulatorUI) to the main computer (VoleMachine), which uses memory, registers, and specific instruction classes to run programs easily.



1st AI Design

Conversation link: https://copilot.microsoft.com/shares/XScXNMF9b33s2SvRheksT

Key Improvements:

- Introduced an InstructionDecoder component to decouple parsing logic from the execution flow.
- Grouped registers into a dedicated RegisterFile class to encapsulate register management.
- Added validation methods and state-dump utilities for enhanced debugging.

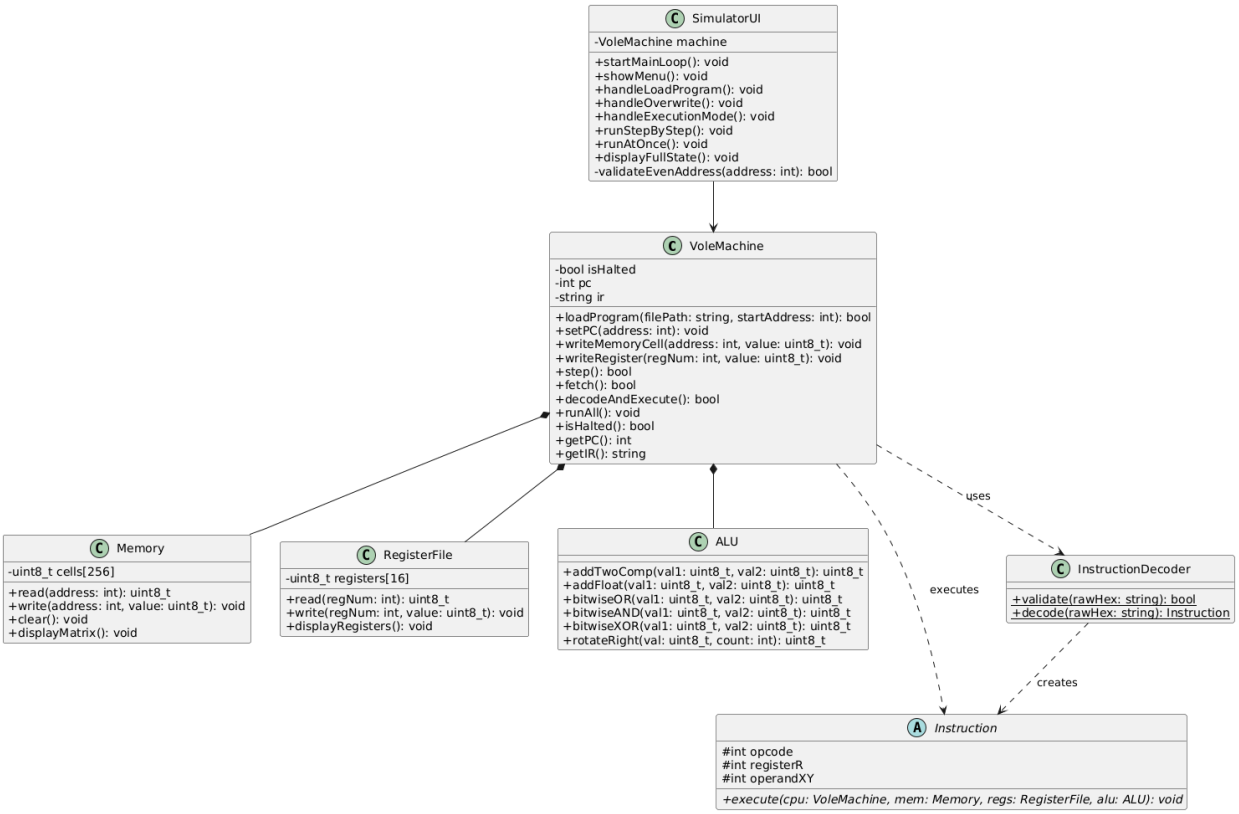


2nd AI Design

Conversation link: https://share.gemini.google/IViSnYbCynPZ

Key Improvements:

- Integrated an ALU (Arithmetic Logic Unit) class to explicitly separate computational logic (two's complement addition, floating-point math, and bitwise operations) from control flow.
- Standardized data types (utilizing fixed-width integer types such as uint8_t).
- Made Instruction an abstract base class to support specialized operation sub-typing.

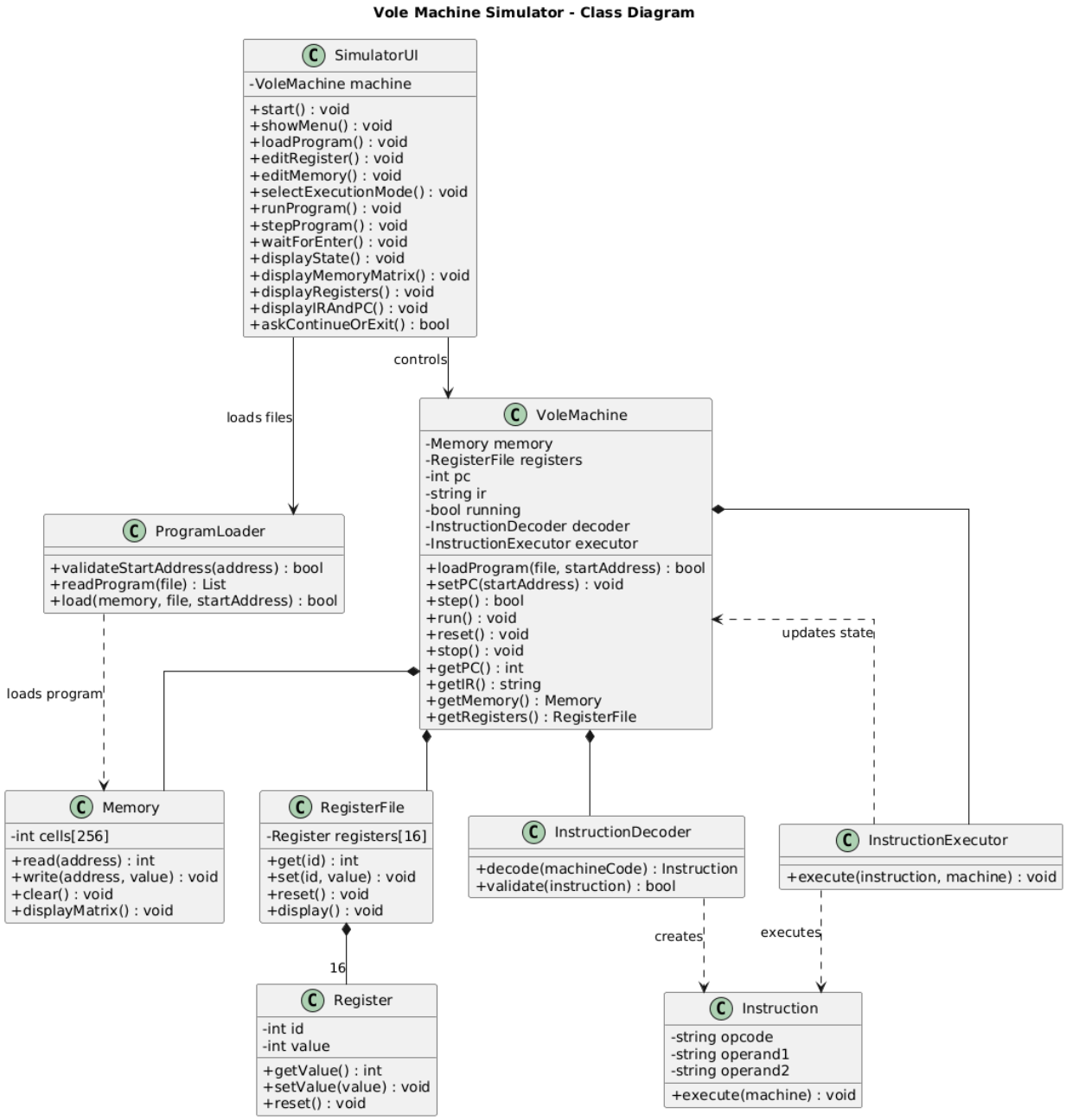


3rd AI Design

Conversation link: https://chatgpt.com/share/6add0fbb-82a8-83ea-b15f-b3cd9cb02250

Key Improvements:

- Separated file parsing concerns into a dedicated ProgramLoader class to follow the Single Responsibility Principle (SRP).
- Added an InstructionExecutor component to delegate execution paths cleanly away from the central machine state.

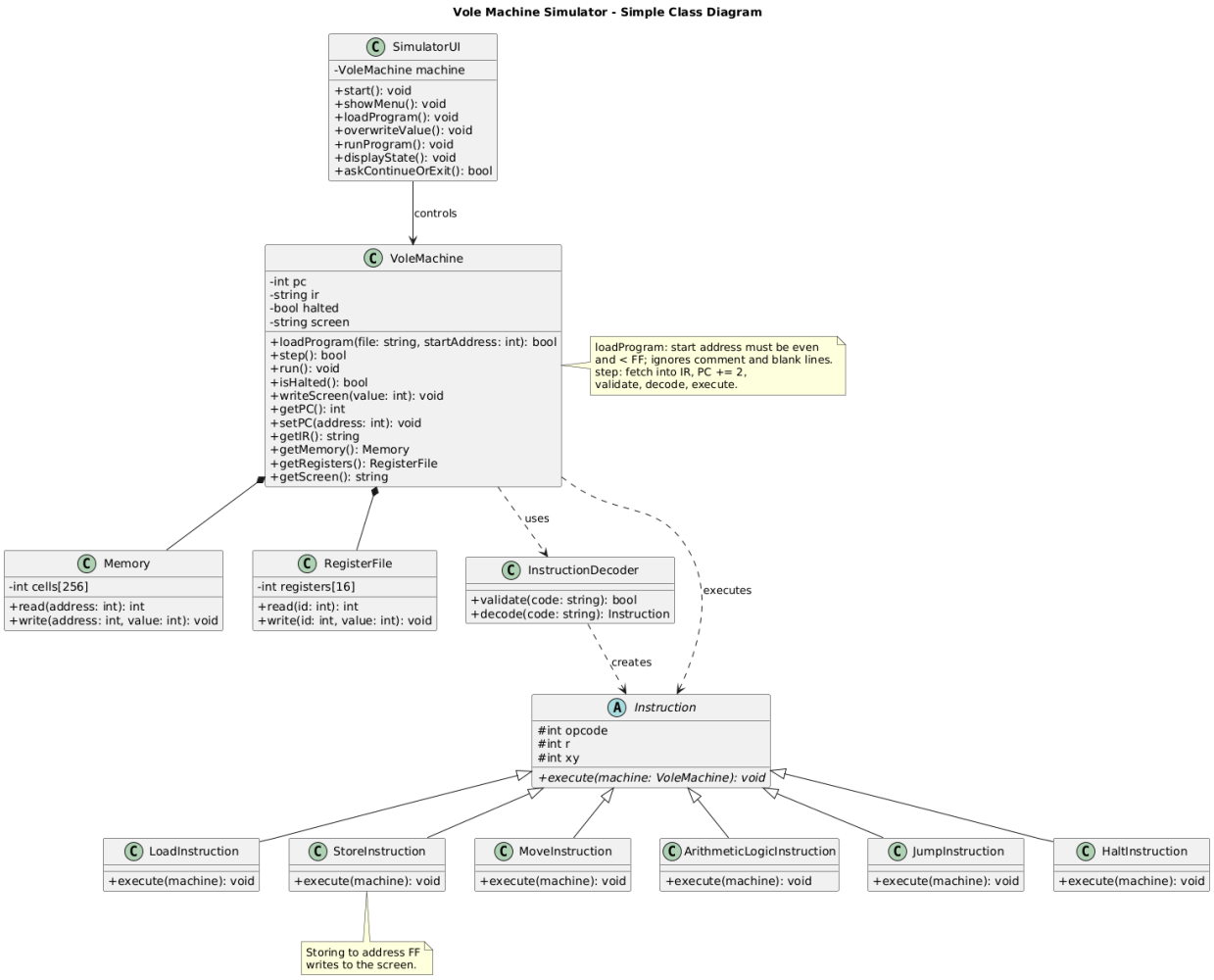


Final Design

Conversation link: https://claude.ai/share/2f7a249d-f69d-4177-bc1f-6f50c31e9bdd

Key Improvements:

- Consolidated and streamlined the architecture for optimal maintainability and clean C++ implementation.
- Implemented robust polymorphic instruction subclasses (LoadInstruction, StoreInstruction, MoveInstruction, ArithmeticLogicInstruction, JumpInstruction, and HaltInstruction) extending an abstract Instruction base class.
- Integrated screen/output handling directly into the system state and refined parser/execution notes for edge cases (e.g., memory bounds and text formatting).



Examples

; Program 1: Addition and Bitwise AND
; This program adds three numbers and applies a Bitwise AND mask.

```
210A
220B
230C
5412
5543
260F
8756
37FF
C000
```

; Program 2: Countdown Loop
; This program runs a loop that counts down until it reaches zero.

```
2000
2103
22FF
31FF
5112
B10E
B006
C000
```

Video link:

https://drive.google.com/file/d/1ponEPLA66F3uJvZX4LV4KhWAh7IExR6_/view?usp=drive_link